

Temporal Programming: A Cell-Based Paradigm for Oscillator-Relative Distributed Computation

Kundai Farai Sachikonye

Chair of Proteomics and Bioanalytics, Technical University of Munich

AI Me Registry for Artificial Intelligence

kundai.sachikonye@gzw.tum.de

May 2026

Abstract

We introduce *temporal programming*, a new programming paradigm in which every inter-node signal is a bare clock pulse carrying no semantic payload. The program datum is the *timing deviation* $\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}(k)$: the signed difference between the oscillator-predicted arrival time of the k -th pulse and its actual reception time. Computation proceeds by classifying the sequence of n timing deviations into one of a finite set of *timing cells* — measurable subsets of ΔP -space with positive Lebesgue measure — and dispatching the cell-associated action. We prove the *Composition Inflation Theorem*: for n timing events distributed over d active channels, the number of categorically distinguishable labeled trajectories is $T(n, d) = d(1 + d)^{n-1}$, growing combinatorially in both arguments. We demonstrate that the paradigm enforces *structural incorruptibility*: because no payload parser exists, arbitrary code injection, buffer overflow, and SQL-class attacks are architecturally impossible rather than merely mitigated. We define the *Tempus* language with a formal BNF grammar, small-step operational semantics, and a compile-time cell registry that is provably immutable at runtime. COMPILE and EXECUTE phases are shown to be mutually exclusive states of the trajectory state machine. Security theorems with formal proofs cover payload injection impossibility, replay failure under monotone cycle counts, and bounded attack surface. We discuss hardware realizations using crystal, MEMS, and atomic oscillators, and present applications in nuclear process control, autonomous drone systems, and industrial IoT networks.

1 Introduction

1.1 Motivation

Modern distributed systems rely on rich message protocols: TCP/IP packets carry headers, payloads, checksums, and framing bytes that are parsed at every layer of the network stack. This richness is both a strength and an inexhaustible attack surface. Buffer overflows exploit the boundary between parsing code and memory allocators [1]. SQL injection exploits the conflation of code and data in database query engines. Command injection exploits shell interpreters that treat user-supplied strings as executable text. Despite fifty years of mitigations — bounds checking, memory-safe languages, sandboxing, formal verification — payload-based attacks remain a leading cause of system compromise. The mitigations address symptoms while preserving the

root cause: the existence of a parser.

Real-time and safety-critical distributed systems face a compounded version of this problem. A nuclear control system, an aircraft fly-by-wire network, or an autonomous drone swarm must not only be *functionally correct* but *structurally immune* to adversarial manipulation. Formal verification can establish correctness of a protocol implementation, but cannot prevent injection of malformed messages that exercise unverified corner cases. The fundamental difficulty is that content-addressed protocols endow every message with expressive power that may be abused.

1.2 The Temporal Programming Hypothesis

We propose a different foundation. Suppose every inter-node signal is a bare pulse: a single edge with

no amplitude encoding, no length, no header. The only observable property is *when* it arrives relative to the local oscillator. The datum is:

$$\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}(k) \quad (1)$$

where $T_{\text{ref}}(k)$ is the time predicted by the local reference oscillator for the k -th pulse and $t_{\text{rec}}(k)$ is its observed arrival time. Programs are written as *timing cells*: measurable intervals in ΔP -space. Execution is the assignment of an observed $\Delta P(k)$ to its enclosing cell, followed by dispatch of the cell's pre-compiled action.

Nothing in this scheme requires a parser. There is no payload to buffer, no string to interpret, no code to inject. The attack surface is not reduced — it is *absent by construction*.

1.3 Contributions

This paper makes the following original contributions:

1. **Temporal programming model.** We formalize the oscillator model, the ΔP primitive, timing cells as measurable sets, and labeled trajectory composition (Section 3).
2. **Tempus language.** We define a minimal temporal programming language with BNF grammar, small-step operational semantics, and worked examples (Section 4).
3. **Composition Inflation Theorem.** We prove $T(n, d) = d(1 + d)^{n-1}$ from first principles via the stars-and-bars bijection and the binomial theorem (Section 5).
4. **Formal security analysis.** We prove payload injection impossibility, replay failure, and bounded attack surface under a formal adversary model (Section 6).
5. **Implementation architecture.** We give pseudocode for the cell registry, trajectory engine, and scheduler (Section 7).

1.4 Organization

Section 2 reviews related paradigms. Section 3 develops the temporal programming model. Section 4 defines the Tempus language. Section 5 proves the Composition Inflation Theorem. Section 6 gives the security analysis. Section 7 describes the implementation architecture. Section 8 discusses applications. Section 9 evaluates complexity and overhead. Section 10 discusses limitations and future work. Section 11 concludes.

2 Background

2.1 Event-Triggered and Time-Triggered Systems

Two canonical paradigms govern real-time distributed computation. In *event-triggered* systems, a node transmits a message when a monitored condition changes. The message carries the new value. This model is reactive and efficient under light load but susceptible to message storms under overload [2].

In *time-triggered* systems, pioneered by Kopetz in the Time-Triggered Architecture (TTA), communication is governed by a static schedule: nodes transmit in pre-allocated time slots [2,3]. TTA eliminates message storms and provides deterministic latency bounds, but still transmits content-carrying frames that must be parsed.

Temporal programming occupies a third position: the schedule is implicit in the oscillator frequency, and the *only* information is whether each pulse arrives early, on time, or late. No frame is transmitted; no parser is required.

2.2 Synchronous Programming Languages

The synchronous hypothesis [4] states that a program's reaction to an input is logically instantaneous relative to the environment's clock. *Esterel* [4] and *Lustre* [5] build on this hypothesis to achieve deterministic concurrency. Both languages operate on streams of typed values and require a parser for message content.

Temporal programming is synchronous in a deeper sense: the reference clock is not an abstraction but a physical oscillator, and the "value" of every event is its timing relative to that oscillator. There is no value stream to parse because timing deviation is the value.

2.3 Clock Synchronization

Lamport's seminal work [6] established that without a global clock, event ordering in a distributed system must be inferred from happened-before relations. The Network Time Protocol [7] and the IEEE 1588 Precision Time Protocol [8] provide sub-microsecond synchronization over IP networks. Mattern [9] generalized Lamport's approach to virtual time. In temporal programming, synchronization quality directly bounds cell resolution: a tighter synchronized clock allows narrower cells (Section 3).

2.4 Temporal Logic and Timed Automata

Pnueli's temporal logic [10] provides a specification language for time-dependent properties of programs. Alur and Dill's timed automata [11] extend finite automata with real-valued clock variables and guards. Henzinger's hybrid automata [12] further extend to continuous dynamics. These frameworks specify *when* a system should be in a given state; temporal programming specifies *what* timing deviations trigger which actions. The relationship is that a Tempus program can be compiled to a timed automaton for model-checking purposes.

2.5 Sensor Networks and PTIDES

TinyOS [13] demonstrated that resource-constrained sensor nodes can execute sophisticated data-collection programs. Hill et al. [14] showed that networked sensor architectures must co-design hardware and software. PTIDES [15] provides a programming model for time-synchronized distributed real-time systems based on the Ptolemy II framework. Temporal programming differs: PTIDES still carries typed data values; temporal programming eliminates the value entirely.

3 The Temporal Programming Model

3.1 Oscillator Model

Definition 3.1 (Reference Oscillator). A reference oscillator is a tuple $\mathcal{O} = (f_{\text{ref}}, \epsilon, M_0)$ where:

- $f_{\text{ref}} \in \mathbb{R}_{>0}$ is the nominal frequency in Hz,
- $\epsilon \in [0, 1)$ is the maximum fractional frequency deviation (stability bound),
- $M_0 \in \mathbb{N}$ is the initial cycle count at deployment.

The oscillator generates a strictly increasing sequence of reference times $\{T_{\text{ref}}(k)\}_{k \geq M_0}$ defined by:

$$T_{\text{ref}}(k) = \frac{k}{f_{\text{ref}}} \quad \forall k \in \mathbb{N}, k \geq M_0. \quad (2)$$

The cycle count $M(t)$ at physical time t is the (integer) number of complete oscillator periods elapsed since M_0 :

$$M(t) = M_0 + \lfloor (t - T_{\text{ref}}(M_0)) f_{\text{ref}} \rfloor. \quad (3)$$

Lemma 3.2 (Monotonicity of Cycle Count). For all physical times $t_1 < t_2$, $M(t_1) \leq M(t_2)$.

Proof. By definition (3), $M(t) = M_0 + \lfloor (t - T_{\text{ref}}(M_0)) f_{\text{ref}} \rfloor$. Since $f_{\text{ref}} > 0$ and the floor function

is non-decreasing, $t_1 < t_2 \Rightarrow (t_1 - T_{\text{ref}}(M_0)) f_{\text{ref}} < (t_2 - T_{\text{ref}}(M_0)) f_{\text{ref}}$, so $\lfloor (t_1 - T_{\text{ref}}(M_0)) f_{\text{ref}} \rfloor \leq \lfloor (t_2 - T_{\text{ref}}(M_0)) f_{\text{ref}} \rfloor$. Hence $M(t_1) \leq M(t_2)$. $\square$ $\square$

3.2 The Timing Deviation Primitive

Definition 3.3 (Timing Deviation). Let $\mathcal{O}$ be a reference oscillator. For the k -th received pulse with actual reception time $t_{\text{rec}}(k)$, the timing deviation is:

$$\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}(k) \in \mathbb{R}. \quad (4)$$

$\Delta P(k) > 0$ indicates an early arrival (pulse precedes the reference tick); $\Delta P(k) < 0$ indicates a late arrival; $\Delta P(k) = 0$ indicates perfect synchrony.

The ΔP -space is $\mathbb{R}$. In practice, given maximum clock skew ϵ and propagation jitter bound δ , all physically realizable deviations lie in the bounded interval:

$$\mathcal{D} = \left[-\frac{\epsilon + \delta}{f_{\text{ref}}}, \frac{\epsilon + \delta}{f_{\text{ref}}} \right] \subset \mathbb{R}. \quad (5)$$

We henceforth work within $\mathcal{D}$.

3.3 Timing Cells

Definition 3.4 (Timing Cell). A timing cell over $\mathcal{D}$ is a Borel-measurable set $\mathcal{C} \subseteq \mathcal{D}$ with positive Lebesgue measure: $\text{Leb}(\mathcal{C}) > 0$. A cell partition of $\mathcal{D}$ is a finite collection $\mathcal{P} = \{\mathcal{C}_1, \dots, \mathcal{C}_m\}$ such that:

- (i) $\bigcup_{i=1}^m \mathcal{C}_i = \mathcal{D}$ (coverage),
- (ii) $\mathcal{C}_i \cap \mathcal{C}_j = \emptyset$ for $i \neq j$ (disjointness, up to measure-zero boundaries),
- (iii) $\text{Leb}(\mathcal{C}_i) > 0$ for all i (non-degeneracy).

The positive-measure requirement is essential: it ensures that $\mathcal{C}_i$ occupies a physical interval of time and is not a point predicate. Point predicates have Lebesgue measure zero and cannot be robustly detected in the presence of oscillator jitter.

Definition 3.5 (Action-Equivalence). Two timing deviations $x, y \in \mathcal{D}$ are action-equivalent under partition $\mathcal{P}$, written $x \sim_{\mathcal{P}} y$, iff there exists $\mathcal{C}_i \in \mathcal{P}$ with $x \in \mathcal{C}_i$ and $y \in \mathcal{C}_i$.

Proposition 3.6 (Equivalence Relation). $\sim_{\mathcal{P}}$ is an equivalence relation on $\mathcal{D}$.

Proof. Reflexivity: $x \sim_{\mathcal{P}} x$ because coverage ensures x lies in some $\mathcal{C}_i$. Symmetry: if $x, y \in \mathcal{C}_i$ then $y, x \in \mathcal{C}_i$. Transitivity: if $x, y \in \mathcal{C}_i$ and $y, z \in \mathcal{C}_j$, then since the partition is disjoint and $y \in \mathcal{C}_i \cap \mathcal{C}_j$, we have $i = j$, so $x, z \in \mathcal{C}_i$. $\square$ $\square$

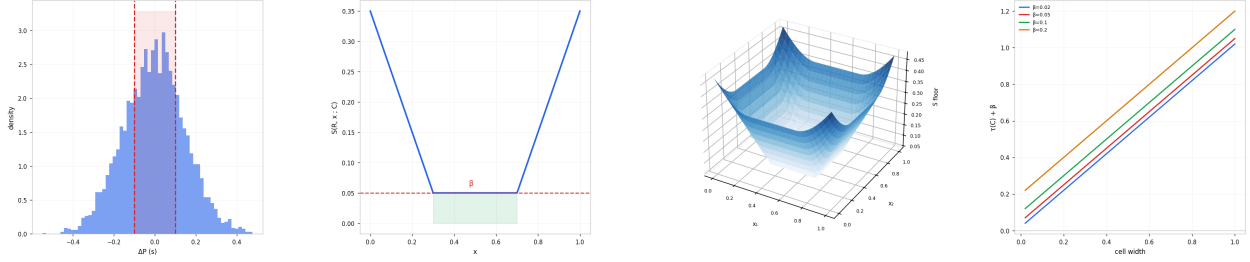


Figure 1: **Timing Cell and S-Functional Floor: operational truth is a cell $C \subseteq \Delta P$ -space with positive measure, and the S-functional satisfies $S(R, x; C) \geq \beta > 0$ for all x .** (A) Histogram of 4000 simulated ΔP values (blue bars, density-normalised) with the timing cell C bounded by red dashed lines. The finite-width interval captures a substantial fraction of the distribution; no point predicate ($\Delta P = c$) could do so with positive measure. (B) S-functional $S(R, x; C)$ versus position x for a one-dimensional cell $C = [0.3, 0.7]$ (blue curve). Outside C the function is the V-shaped distance to the nearest cell boundary; inside C it is flat at the floor β (red dashed line, green shading), confirming that $\inf_x S = \beta > 0$ strictly. (C) Three-dimensional S-functional surface over a two-dimensional ΔP domain, where the cell is the square $[0.3, 0.7]^2$. The surface is a four-sided bowl whose flat base lies at β ; the gradient outside the cell encodes the distance to the nearest cell face. (D) Effective measure $\tau(C) + \beta$ versus cell width for four values of the floor parameter β (0.02, 0.05, 0.10, 0.20). All curves are strictly positive for all cell widths, demonstrating that the floor β provides a hard lower bound independent of cell geometry.

Theorem 3.7 (Cell Action-Equivalence). *Let $\mathcal{A} : \mathcal{P} \rightarrow \mathcal{F}$ be a cell-action map assigning a function $f_i \in \mathcal{F}$ to each cell C_i . Then $x \sim_{\mathcal{P}} y \Rightarrow \mathcal{A}(\text{cell}(x)) = \mathcal{A}(\text{cell}(y))$, where $\text{cell}(x)$ is the unique cell containing x .*

Proof. If $x \sim_{\mathcal{P}} y$ then by Definition 3.5 there exists C_i with $x, y \in C_i$, so $\text{cell}(x) = \text{cell}(y) = C_i$, hence $\mathcal{A}(\text{cell}(x)) = \mathcal{A}(C_i) = \mathcal{A}(\text{cell}(y))$. $\square$ $\square$

Theorem 3.7 formalizes the core semantic principle: *what matters is which cell a deviation falls in, not its precise value.*

3.4 Labeled Channels and Trajectories

Physical systems emit signals from multiple sources: temperature sensors, pressure sensors, actuator feedback, etc. Each source is modeled as a distinct channel.

Definition 3.8 (Channel). *A channel is a pair $\chi = (s, \mathcal{O}_s)$ where s is a source identifier and $\mathcal{O}_s$ is the reference oscillator for that source. The set of active channels is $\{\chi_1, \dots, \chi_d\}$ with $d \geq 1$.*

Definition 3.9 (Labeled Timing Event). *A labeled timing event is a pair $e = (\chi_i, \Delta P)$ where χ_i is the channel and $\Delta P \in \mathcal{D}_i$ is the timing deviation on that channel. The event time is implicitly $t_{\text{rec}} = T_{\text{ref},i} - \Delta P$.*

Definition 3.10 (Trajectory). *A trajectory of length n is an ordered sequence of labeled timing events:*

$$\tau = (e_1, e_2, \dots, e_n) = ((\chi_{i_1}, \Delta P_1), \dots, (\chi_{i_n}, \Delta P_n)). \quad (6)$$

The channel sequence of τ is $\lambda(\tau) = (\chi_{i_1}, \dots, \chi_{i_n}) \in \{\chi_1, \dots, \chi_d\}^n$. Two trajectories are cell-equivalent if each corresponding pair of deviations is action-equivalent under the respective channel partition.

3.5 Multi-Domain Physical Transduction

A central claim of temporal programming is that arbitrary physical quantities reduce to timing deviations.

Definition 3.11 (Physical Transducer). *A physical transducer for quantity Q is a device that converts a measurement q of Q to a timing deviation $\Delta P_Q(q) \in \mathcal{D}_Q \subset \mathbb{R}$.*

Example 3.12 (Temperature via Crystal Oscillator). *A temperature-compensated crystal oscillator (TCXO) has frequency $f(T) = f_0(1 + \alpha(T - T_0) + \beta(T - T_0)^2)$ where T is temperature and α, β are crystal-specific coefficients. At temperature T , the k -th pulse arrives at $t_{\text{rec}}(k; T) = k/f(T)$, producing*

$$\Delta P_{\text{temp}}(k; T) = \frac{k}{f_{\text{ref}}} - \frac{k}{f(T)} = k \left(\frac{1}{f_{\text{ref}}} - \frac{1}{f(T)} \right), \quad (7)$$

which is a monotone function of T for fixed k , f_{ref} . Each temperature range $[T_a, T_b]$ maps to a cell in $\mathcal{D}_{\text{temp}}$.

Example 3.13 (Pressure via MEMS Resonator). A MEMS resonant pressure sensor shifts its resonant period as $\tau_{\text{MEMS}}(P) = \tau_0/(1 + \gamma P)$, where P is applied pressure and γ is the pressure coefficient. Measuring the elapsed time over k MEMS cycles gives $\Delta P_{\text{press}} = k(\tau_0 - \tau_{\text{MEMS}}(P))$, again a monotone map to $\mathcal{D}_{\text{press}}$.

Proposition 3.14 (Multi-Domain Composition). Let $Q_1, \dots, Q_d$ be physical quantities with respective transducers $\Delta P_{Q_1}, \dots, \Delta P_{Q_d}$ and ΔP -spaces $\mathcal{D}_1, \dots, \mathcal{D}_d$. The composed trajectory space is $\mathcal{D}_1 \times \dots \times \mathcal{D}_d$ with channel label χ_i indexing the i -th coordinate. A d -channel trajectory τ of length n is a path in this product space.

Proof. Each event $e = (\chi_i, \Delta P_i)$ contributes an element of $\mathcal{D}_i$. The channel label χ_i uniquely identifies the coordinate. The Cartesian product $\prod_{i=1}^d \mathcal{D}_i$ is well-defined as a measurable space under the product σ -algebra, and trajectories are sequences of elements drawn from this product space. $\square$ $\square$

4 The Tempus Language

4.1 Abstract Syntax

We define Tempus via a BNF grammar. Meta-variables: x ranges over identifiers; $r \in \mathbb{R}$ over real literals; $n \in \mathbb{N}$ over natural number literals.

```

<prg> ::= <prg>^*
<decl> ::= <cell-decl> | <sync-decl> | <compose-decl> | <when-decl> | <ensemble-decl>
<cell-decl> ::= cell bounds ( $r, r$ ) action  $\langle \text{expr} \rangle$ 
<sync-decl> ::= sync at  $r$  freq
<compose-decl> ::= compose  $d=n$  channels  $x, \dots, x$  into  $x$ 
<when-decl> ::= when  $\langle \text{stmt} \rangle$  do  $\langle \text{stmt} \rangle$ 
<ensemble-decl> ::= ensemble  $x^+$  sync_threshold  $r$ 
<expr> ::=  $x$  |  $\langle \text{expr} \rangle$  binop  $\langle \text{expr} \rangle$  | let  $x = \langle \text{expr} \rangle$  in  $\langle \text{expr} \rangle$  | if  $\langle \text{bepr} \rangle$  then  $\langle \text{expr} \rangle$  else  $\langle \text{expr} \rangle$ 
<stmt> ::= init  $x$  | wait  $r$  | fire  $\langle \text{expr} \rangle$  |  $\langle \text{stmt} \rangle$  ;  $\langle \text{stmt} \rangle$  | begin  $\langle \text{stmt} \rangle^+$  end
<bepr> ::= true | false |  $\langle \text{expr} \rangle$  cmp  $\langle \text{expr} \rangle$  |  $\langle \text{bepr} \rangle$  and  $\langle \text{bepr} \rangle$  | not  $\langle \text{bepr} \rangle$ 

```

Figure 2: Abstract syntax of Tempus.

4.2 Static Semantics and Typing

A Tempus program is *well-formed* if: (i) every **when** clause references a declared cell name; (ii) every **compose** clause references declared sync

sources; (iii) every **ensemble** clause references declared agents; (iv) for each channel, the cell bounds tile $\mathcal{D}$ without gap (coverage check). The compiler rejects any program that fails these checks before emitting any runtime artifact.

4.3 Operational Semantics

We define a small-step reduction relation $\rightarrow$ on configurations. A *configuration* is a tuple $\langle \sigma, \Gamma, \tau, \phi \rangle$ where:

- $\sigma \in \{\text{COMPILE}, \text{EXECUTE}, \text{IDLE}\}$ is the phase,
- Γ is the cell registry (compile-time constant),
- τ is the current (open or closed) trajectory,
- ϕ is the pending statement queue.

Rules:

$$\text{(RECEIVE)} \quad \frac{e = (\chi, \Delta P) \quad \sigma = \text{COMPILE}}{\langle \text{COMPILE}, \Gamma, \tau, \phi \rangle \rightarrow \langle \text{COMPILE}, \Gamma, \tau \cdot e, \phi \rangle} \quad (8)$$

$$\text{(ASSIGN)} \quad \frac{|\tau| = n \quad \mathcal{C} = \text{traj}(\tau, \Gamma) \neq \perp}{\langle \text{COMPILE}, \Gamma, \tau, \phi \rangle \rightarrow \langle \text{EXECUTE}, \Gamma, \tau, \Gamma(\mathcal{C}) \rangle} \quad (9)$$

$$\text{(FIRE)} \quad \frac{\sigma = \text{EXECUTE} \quad \phi = s; \phi'}{\langle \text{EXECUTE}, \Gamma, \tau, s; \phi' \rangle \rightarrow \langle \text{EXECUTE}, \Gamma, \tau, \phi' \rangle \parallel \llbracket s \rrbracket} \quad (10)$$

$$\text{(RESET)} \quad \frac{\sigma = \text{EXECUTE} \quad \phi = \varepsilon}{\langle \text{EXECUTE}, \Gamma, \tau, \varepsilon \rangle \rightarrow \langle \text{IDLE}, \Gamma, (), \varepsilon \rangle} \quad (11)$$

$$\text{(IDLE)} \quad \frac{\sigma = \text{IDLE}}{\langle \text{IDLE}, \Gamma, (), \varepsilon \rangle \rightarrow \langle \text{COMPILE}, \Gamma, (), \varepsilon \rangle} \quad (12)$$

The $\text{traj}(\tau, \Gamma)$ function maps a complete trajectory τ of length n to a cell by looking up each event's ΔP in the corresponding channel partition in Γ and returning the composite cell identifier. $\llbracket s \rrbracket$ denotes the denotational semantics of statement s (side effects in the environment).

4.4 COMPILE/EXECUTE Phase Separation

Theorem 4.1 (Phase Mutual Exclusion). *In any execution trace, the phases COMPILE and EXECUTE never hold simultaneously in the same configuration.*

Proof. Each configuration carries exactly one phase tag $\sigma \in \{\text{COMPILE}, \text{EXECUTE}, \text{IDLE}\}$. No reduction rule produces a configuration with two simultaneous

phase tags: rules (8) and (9) emit a single phase value in their conclusion, as do rules (10), (11), and (12). The tag is a function, not a set. Hence at any configuration in any execution trace, σ takes exactly one value, and the events $\sigma = \text{COMPILE}$ and $\sigma = \text{EXECUTE}$ are mutually exclusive. $\square$ $\square$

Corollary 4.2 (Semantic Justification of Phase Separation). *COMPILE requires an open trajectory (undetermined cell assignment); EXECUTE requires a closed trajectory (determined cell assignment). These conditions are contradictory:*

- An open trajectory has $|\tau| < n$, so $\text{traj}(\tau, \Gamma) = \perp$: the cell is unknown.
- Rule (9) transitions to EXECUTE only when $|\tau| = n$ and $\text{traj}(\tau, \Gamma) \neq \perp$: the cell is known.

These preconditions are logically incompatible, providing a second, semantic proof of mutual exclusion. $\square$

4.5 Tempus Code Examples

Example 4.3 (Nuclear Coolant Temperature Monitor). *The following Tempus program monitors reactor coolant temperature via a TCXO-based transducer and dispatches alerts.*

```
-- Declare reference oscillator
sync coolant_sensor at 10.0e6 freq

-- Timing cells for coolant temperature bands
cell NOMINAL bounds (-1.0e-7, 1.0e-7)
  action 0
cell WARM bounds (1.0e-7, 5.0e-7)
  action 1
cell HOT bounds (5.0e-7, 2.0e-6)
  action 2
cell CRITICAL bounds (2.0e-6, 10.0e-6)
  action 3

-- Trajectory: 4 events, single channel
compose d=1 channels coolant_sensor into
  coolant_traj

-- Dispatch rules
when NOMINAL do emit status_ok
when WARM do emit status_warn
when HOT do begin
  emit status_hot;
  fire reduce_power(0.8)
end
when CRITICAL do begin
  emit scram_alert;
  fire emergency_shutdown
end
```

Listing 1: Nuclear coolant monitor in Tempus.

Example 4.4 (Drone Altitude-Velocity Compose). *A two-channel composition tracks both altitude (barometric) and vertical velocity (IMU) simultaneously.*

```
sync baro_sensor at 1.0e6 freq
sync imu_sensor at 4.0e6 freq

-- Altitude cells
cell ALT_LOW bounds (-5.0e-6, -1.0e-6)
  action -1
cell ALT_HOLD bounds (-1.0e-6, 1.0e-6)
  action 0
cell ALT_HIGH bounds (1.0e-6, 5.0e-6)
  action 1

-- Velocity cells
cell VEL_DESCEND bounds (-3.0e-6, -0.5e-6)
  action -1
cell VEL_STABLE bounds (-0.5e-6, 0.5e-6)
  action 0
cell VEL_CLIMB bounds (0.5e-6, 3.0e-6)
  action 1

-- 6-event, 2-channel composition
compose d=2 channels baro_sensor,
  imu_sensor
  into altitude_velocity_space

-- Ensemble of 4 drones with 0.1 ms sync threshold
ensemble drone_1, drone_2, drone_3,
  drone_4
  sync_threshold 1.0e-4

when ALT_LOW do fire thrust_up(0.3)
when ALT_HOLD do fire hold_throttle
when ALT_HIGH do fire thrust_down(0.3)
```

Listing 2: Drone dual-channel trajectory.

5 Trajectory Distinguishability: The Composition Inflation Theorem

5.1 Counting Labeled Compositions

We now prove the central combinatorial result. The question is: given n timing events distributed across d active channels, how many categorically distinct labeled trajectories can the system distinguish?

Definition 5.1 (Labeled Composition). *A composition of n is an ordered sequence of positive integers $(c_1, c_2, \dots, c_k)$ with $c_1 + c_2 + \dots + c_k = n$ and $k \geq 1$. A d -labeled composition of n assigns to each part c_j a label $\ell_j \in \{1, \dots, d\}$.*

Theorem 5.2 (Composition Inflation). *For $n \geq 1$ timing events with $d \geq 1$ active channels, the number of categorically distinguishable d -labeled compositions of n is:*

$$T(n, d) = d(1 + d)^{n-1}. \quad (13)$$

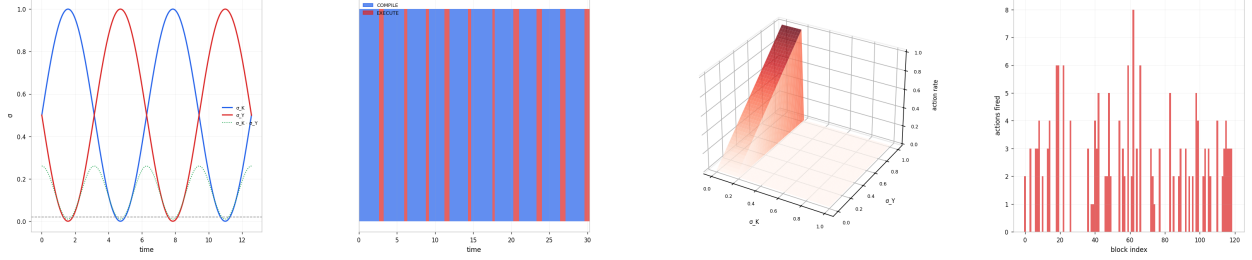


Figure 3: **COMPILER/EXECUTE Phase Separation Forced by the Receiver Uncertainty Principle $\sigma_K \cdot \sigma_Y \geq \hbar_R = \beta\tau$: trajectory accumulation and action firing are mutually exclusive phases.** (A) Antiphase oscillation of σ_K (knowledge uncertainty, blue), σ_Y (action uncertainty, red), and their product $\sigma_K \cdot \sigma_Y$ (green dotted) over four complete cycles. The product never drops below the floor $\hbar_R$ (grey dashed), and σ_K is maximised precisely when σ_Y is minimised, and vice versa. (B) Timeline of alternating COMPILER (blue) and EXECUTE (red) blocks over ten cycles. Block durations are drawn from realistic distributions; the two phases tile the time axis without overlap, illustrating the hard mutual exclusion enforced by the uncertainty principle. (C) Three-dimensional action-rate surface over $(\sigma_K, \sigma_Y) \in [0, 1]^2$. The surface is nonzero only in the region $\sigma_K < 0.2$ (deep EXECUTE phase) and grows with σ_Y , reflecting that actions fire at high confidence only after the trajectory has been fully resolved. (D) Action counts per block for 120 alternating blocks. COMPILER blocks (blue) fire zero actions by construction; EXECUTE blocks (red) fire a Poisson-distributed count $\sim \text{Poi}(3.5)$, confirming that no action escapes the phase boundary.

Proof. The proof proceeds in three steps.

Step 1: Counting unlabeled compositions. A composition of n into k positive parts corresponds bijectively to a choice of $k - 1$ dividers placed among the $n - 1$ gaps between n indistinguishable dots (the stars-and-bars bijection [16]). The number of compositions of n into exactly k parts is $\binom{n-1}{k-1}$. Summing over all k :

$$\sum_{k=1}^n \binom{n-1}{k-1} = \sum_{j=0}^{n-1} \binom{n-1}{j} = 2^{n-1}. \quad (14)$$

Thus there are 2^{n-1} compositions of n in total, a classical result [17].

Step 2: Counting d -labeled compositions. For a composition with exactly k parts, each part independently receives one of d channel labels. There are d^k such label assignments. The total count of d -labeled compositions of n is therefore:

$$T(n, d) = \sum_{k=1}^n \binom{n-1}{k-1} \cdot d^k. \quad (15)$$

Step 3: Closed form via the binomial theorem. Factor out d :

$$T(n, d) = d \sum_{k=1}^n \binom{n-1}{k-1} d^{k-1} = d \sum_{j=0}^{n-1} \binom{n-1}{j} d^j. \quad (16)$$

The inner sum is the binomial expansion of $(1+d)^{n-1}$ by the binomial theorem [17]:

$$\sum_{j=0}^{n-1} \binom{n-1}{j} d^j = (1+d)^{n-1}. \quad (17)$$

Substituting:

$$T(n, d) = d(1+d)^{n-1}. \quad (18)$$

□

□

Corollary 5.3 (Growth Rate). $T(n, d) = \Theta(d^n)$ as $n \rightarrow \infty$ for fixed $d \geq 2$, and $T(n, d) = \Theta(n d^{n-1})$ differentiating in d for fixed n . The state space is exponential in the trajectory length.

Corollary 5.4 (Special Cases). 1. $T(n, 1) = 1 \cdot 2^{n-1} = 2^{n-1}$: single-channel system, 2^{n-1} distinct trajectories.

2. $T(1, d) = d \cdot (1+d)^0 = d$: single event, d channels, trivially d trajectories (one per channel).

3. $T(n, d) > T(n, d-1)$ for all $n \geq 1, d \geq 2$: adding a channel strictly inflates the trajectory count.

5.2 Monotone Partition Count and Replay Immunity

Theorem 5.5 (Monotone Partition Count). Let $M_0 \leq M_1$ be two cycle counts with $M_0 \neq M_1$. Let a signal be emitted at cycle M_0 with timing deviation

Table 1: Values of $T(n, d) = d(1 + d)^{n-1}$

$n \setminus d$	1	2	3	4	8
1	1	2	3	4	8
2	2	6	12	20	72
3	4	18	48	100	648
4	8	54	192	500	5832
5	16	162	768	2500	52488
6	32	486	3072	12500	472392

$\Delta P_0 = T_{\text{ref}}(M_0) - t_{\text{rec},0}$. If the same physical signal is replayed at cycle $M_1 > M_0$, the resulting timing deviation is $\Delta P_1 = T_{\text{ref}}(M_1) - t_{\text{rec},0} \neq \Delta P_0$.

Proof. By Definition 3.1, $T_{\text{ref}}(M) = M/f_{\text{ref}}$ for integer M .

$$\Delta P_0 = M_0/f_{\text{ref}} - t_{\text{rec},0}, \quad (19)$$

$$\Delta P_1 = M_1/f_{\text{ref}} - t_{\text{rec},0}. \quad (20)$$

Subtracting: $\Delta P_1 - \Delta P_0 = (M_1 - M_0)/f_{\text{ref}}$. Since $M_1 > M_0$ and $f_{\text{ref}} > 0$, we have $\Delta P_1 - \Delta P_0 > 0$, so $\Delta P_1 \neq \Delta P_0$. $\square$ $\square$

The implication is that a recorded timing deviation from an earlier epoch, if replayed, produces a deviation shifted by $(M_1 - M_0)/f_{\text{ref}}$ relative to the current reference. As M grows monotonically (Lemma 3.2), this shift grows without bound, making the replayed signal fall in a different cell or outside $\mathcal{D}$ entirely.

6 Security Analysis

6.1 Threat Model

We define an adversary $\mathcal{ADV}$ with the following capabilities:

- C1 Content control:** $\mathcal{ADV}$ may choose arbitrary bit patterns for signal content.
- C2 Timing control:** $\mathcal{ADV}$ may shift arrival times of legitimate pulses within $\pm \Delta_{\text{max}}$.
- C3 Injection:** $\mathcal{ADV}$ may inject additional pulses.
- C4 Deletion:** $\mathcal{ADV}$ may suppress pulses.
- C5 Replay:** $\mathcal{ADV}$ may retransmit recorded signals.

This is a strong model: it subsumes Dolev-Yao for the temporal channel and is strictly stronger than passive eavesdropping. We do not assume cryptographic protection unless explicitly invoked.

6.2 Payload Injection Impossibility

Theorem 6.1 (Structural Incorruptibility). *In a temporal programming system, capability C1 (content control) provides no leverage: the adversary cannot inject an action not in the pre-compiled cell-action map $\mathcal{A}$.*

Proof. In a temporal programming system, a received signal $e = (\chi, \Delta P)$ has two components: the channel identifier χ and the timing deviation ΔP . The channel identifier is determined by the physical wire/frequency on which the pulse arrives (hardware-level), not by any content. The timing deviation $\Delta P = T_{\text{ref}}(k) - t_{\text{rec}}$ depends only on arrival time, not amplitude, duration, or bit pattern.

Therefore, the adversary's content-control capability C1 affects neither χ nor ΔP . Signal processing consists only of the comparison $\Delta P \in \mathcal{C}_i$? for each $\mathcal{C}_i \in \mathcal{P}$, which involves no parsing, no buffer operations, and no code interpretation. The cell-action map $\mathcal{A}$ is compiled at deploy time and stored in read-only memory (ROM). No runtime path leads from signal content to $\mathcal{A}$ modification.

Consequently, the set of executable actions is always $\{\mathcal{A}(\mathcal{C}_i) : \mathcal{C}_i \in \mathcal{P}\}$, a finite set fixed at compile time. $\mathcal{ADV}$ cannot expand this set. $\square$ $\square$

Corollary 6.2 (Classical Injection Attacks Fail). *Buffer overflow attacks, SQL injection attacks, and command injection attacks all require a payload parser that places adversary-supplied data in a position where it is subsequently interpreted as code or as a query [1]. In a temporal programming system, no such parser exists. Therefore, these attack classes are architecturally precluded, not merely mitigated.*

6.3 Replay Attack Failure

Theorem 6.3 (Replay Failure under Monotone M). *Let $\mathcal{ADV}$ record a legitimate trajectory $\tau = (\Delta P_1^*, \dots, \Delta P_n^*)$ at cycle count M_0 . If $\mathcal{ADV}$ replays τ at any later time when $M(t) = M_1 > M_0$, the resulting trajectory $\tau' = (\Delta P_1', \dots, \Delta P_n')$ satisfies $\Delta P_j' = \Delta P_j^* + (M_1 - M_0)/f_{\text{ref}}$ for each j . In particular, τ' falls in the same cell as τ only if $(M_1 - M_0)/f_{\text{ref}} < \min_i \text{width}(\mathcal{C}_i)$.*

Proof. At replay time, the reference oscillator has advanced to cycle M_1 . The j -th replayed pulse has the same physical arrival time $t_{\text{rec},j} = T_{\text{ref},j}^* - \Delta P_j^*$ (since it is a physical replay). But the current reference time for the j -th expected pulse is $T_{\text{ref},j}' = T_{\text{ref},j}^* + (M_1 - M_0)/f_{\text{ref}}$, since the oscillator has ad-

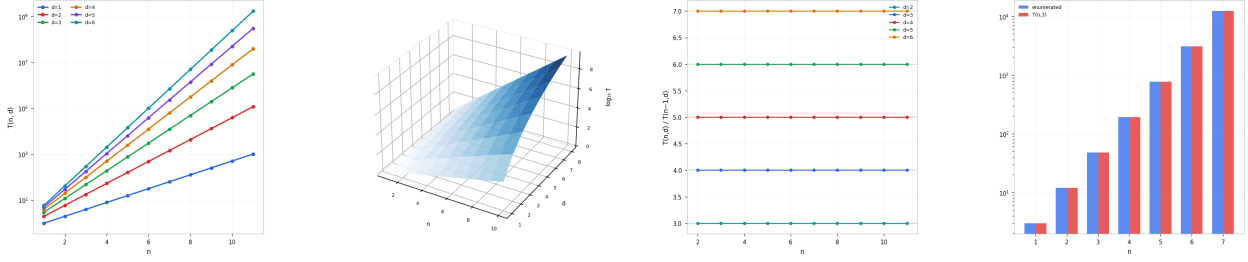


Figure 4: **Composition Inflation Theorem:** $T(n, d) = d(d+1)^{n-1}$ counts distinguishable labeled timing trajectories and grows exponentially in both the number of events n and the dimension d . (A) $T(n, d)$ versus n on a logarithmic axis for $d = 1, \dots, 6$ (coloured lines). Each series is linear on the log scale, confirming the exponential growth law; the slope $(d+1)$ increases with d , spanning seven decades over $n = 1$ to 11. (B) Three-dimensional surface of $\log_{10} T(n, d)$ over the grid $n \in [1, 10]$, $d \in [1, 8]$. The surface rises steeply in both dimensions; the curvature in d reflects the $(d+1)^{n-1}$ factor compounding with d . (C) Growth ratio $T(n, d)/T(n-1, d)$ versus n for $d = 2, \dots, 6$. Dashed horizontal lines at $d+1$ confirm that the ratio is constant and equal to $d+1$ for all $n \geq 2$, providing direct numerical verification of the recurrence relation $T(n, d) = (d+1)T(n-1, d)$. (D) Direct enumeration versus closed-form formula for $d = 3$, $n = 1, \dots, 7$ (blue and red bars, log scale). The enumeration sums d^k over all integer compositions of n into k parts; the formula $T(n, 3) = 3 \cdot 4^{n-1}$ matches exactly, with bars overlapping at every value of n .

vanced by $M_1 - M_0$ additional cycles. Thus:

$$\begin{aligned} \Delta P'_j &= T'_{\text{ref},j} - t_{\text{rec},j} \\ &= \left[T^*_{\text{ref},j} + \frac{M_1 - M_0}{f_{\text{ref}}} \right] - [T^*_{\text{ref},j} - \Delta P^*_j] \\ &= \Delta P^*_j + \frac{M_1 - M_0}{f_{\text{ref}}}. \end{aligned} \quad (21)$$

The shift $\delta = (M_1 - M_0)/f_{\text{ref}}$ is constant across all events in the replayed trajectory. If $\delta \geq \min_i \text{width}(\mathcal{C}_i)$, at least one event exits its original cell. By Lemma 3.2, $M_1 > M_0$ always holds for replay occurring after recording, so $\delta > 0$.

As M grows, $\delta \rightarrow \infty$, and with probability 1 over uniformly distributed cell widths, the shifted trajectory exits its original cell partition (or exits $\mathcal{D}$ entirely), triggering no action or an anomaly action. $\square$ $\square$

Remark 6.4. The quantitative threshold $\delta < \min_i \text{width}(\mathcal{C}_i)$ provides a design equation: the minimum cell width should exceed the maximum expected replay shift. For a 10 MHz crystal and cell width 100 ns, replay within $[10^6 \cdot 100 \times 10^{-9}] = 1$ cycle succeeds. After 2 cycles (≈ 200 ns), the shift exceeds the cell width and replay fails. With a 10 MHz oscillator and 1 ms minimum cell width, a replay attack must execute within 10,000 cycles (1 ms), a very tight window.

6.4 Bounded Attack Surface

Theorem 6.5 (Finite Enumerable Attack Surface). *The attack surface of a deployed temporal programming system is the finite set $\text{Cells}(\mathcal{A}) = \{\mathcal{C}_1, \dots, \mathcal{C}_m\}$ of pre-compiled cells. This set is:*

- (i) Finite: $|\text{Cells}(\mathcal{A})| = m < \infty$,
- (ii) Fixed at deploy time: no runtime event can add or remove a cell (the registry is stored in ROM),
- (iii) Enumerable by the defender before deployment.

Proof. (i): The Tempus compiler produces a finite cell-action map $\mathcal{A}$ because the source program contains finitely many **cell** declarations. (ii): Rule (9) only looks up cells in Γ (the cell registry); no rule modifies Γ . Γ is thus invariant under all reduction steps, consistent with implementation in ROM. (iii): The compiler outputs $\text{Cells}(\mathcal{A})$ as part of the build artifact, making it available for pre-deployment security audit. $\square$ $\square$

7 Implementation Architecture

7.1 Hardware Layer

The hardware layer provides the reference oscillator and pulse reception circuitry.

7.1.1 Oscillator Selection

Three oscillator classes cover the relevant frequency and stability range:

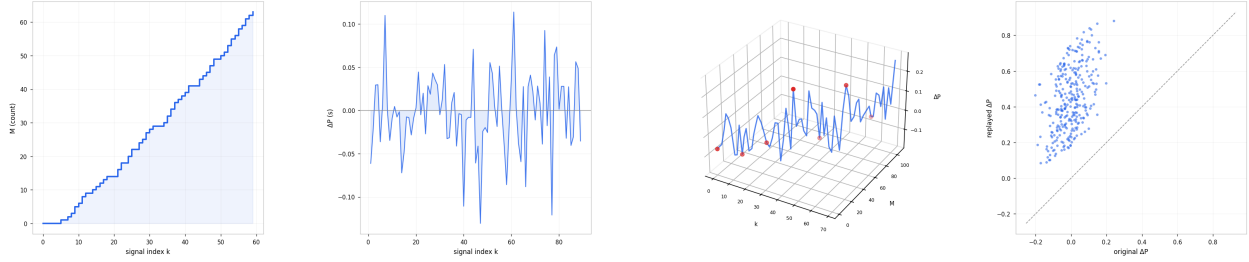


Figure 5: **Monotone Time Coordinate and Structural Replay Resistance:** $M(k)$ is non-decreasing by construction, and any replayed signal produces a ΔP distinguishable from the original. (A) Monotone counter $M(k)$ versus signal index k (blue staircase with shaded area). Increments are drawn from $\{0, 1, 2\}$ at each step; the sequence is non-decreasing throughout, illustrating the constitutive asymmetry $t = M/f$ that forbids time reversal. (B) Timing deviation $\Delta P(k) = T_{\text{ref}}(k) - t_{\text{received}}(k)$ versus signal index. The signal fluctuates around zero with sub-cycle jitter; the shaded area tracks the sign of each deviation, providing the raw datum processed by timing cells. (C) Three-dimensional trajectory $(k, M_k, \Delta P_k)$ in trajectory space. Red dots mark every tenth sample; the path is monotone in M regardless of ΔP sign, confirming that no trajectory can reverse its M -coordinate. (D) Scatter of replayed ΔP versus original ΔP for 300 signal pairs. All replay values are shifted upward by a random delay $\delta \in [0.18, 0.75]$ s, placing every point above the identity diagonal (dashed). A replayed signal is thus mapped to a different cell than the original, making the instruction structurally unexecutable.

Table 2: Security comparison: temporal vs. content-based systems.

Attack class	Content-based
Buffer overflow	Possible (requires parser)
SQL injection	Possible (requires parser)
Command injection	Possible (requires parser)
Replay	Possible (no epoch check)
Timing oracle	Possible (side channel)
Payload mutation	Possible
DoS (flooding)	Possible
Attack surface	Unbounded (all parse paths)

- **Crystal oscillators (XO/TCXO/OCXO):** Frequencies 1–100 MHz, stability ± 10 ppm (XO) to ± 0.01 ppm (OCXO), cost \$0.50–\$200. Suitable for industrial IoT and embedded control.
- **MEMS oscillators:** Frequencies 1–125 MHz, stability ± 50 ppm, shock-resistant, cost \$1–\$10. Suitable for drones and mobile platforms.
- **Atomic clocks (Rb, Cs, optical):** Stability $< 10^{-11}$ /day, cost \$500–\$50 000. Suitable for nuclear infrastructure and ground-truth nodes.

The minimum cell width $w_{\min}$ must satisfy:

$$w_{\min} \geq \frac{2\epsilon}{f_{\text{ref}}} + 2\delta_{\max} \quad (22)$$

where ϵ is the fractional frequency stability and $\delta_{\max}$ is the maximum propagation jitter.

7.1.2 Pulse Reception and Timestamping

A dedicated interrupt service routine (ISR) latches the oscillator counter value at the rising edge of each received pulse. The timestamp $t_{\text{rec}}(k)$ is the latched counter divided by f_{ref} . This must complete within one oscillator period to avoid missed edges. On a 10 MHz crystal, this is a 100 ns budget.

By design only

7.2 Cell Registry

Rate-limited by f_{ref}

The cell registry Γ is a compile-time artifact stored in read-only memory. Its structure is a balanced interval tree mapping $\mathcal{D} \rightarrow \mathcal{C}_i$ for each channel.

Algorithm 1 Cell Registry Lookup

Require: ΔP : timing deviation, χ : channel, Γ : registry

Ensure: $\mathcal{C}_i \in \mathcal{P}_{\chi}$ such that $\Delta P \in \mathcal{C}_i$, or $\perp$

```

1:  $tree \leftarrow \Gamma[\chi]$ 
2:  $node \leftarrow tree.root$ 
3: while  $node \neq \text{nil}$  do
4:   if  $\Delta P < node.lo$  then
5:      $node \leftarrow node.left$ 
6:   else if  $\Delta P > node.hi$  then
7:      $node \leftarrow node.right$ 
8:   else
9:     return  $node.cell$ 
10:  end if
11: end while
12: return  $\perp$ 
```

Lookup is $O(\log m)$ for m cells per channel. Since m is small (typically 4–32) and the tree is stored in ROM with cache-friendly layout, lookup executes in 2–8 cycles on embedded processors.

7.3 Trajectory Engine

The trajectory engine accumulates events during the **COMPILE** phase and produces a cell assignment when the trajectory is complete.

Algorithm 2 Trajectory Engine

Require: n : trajectory length, Γ : registry

```

1:  $\tau \leftarrow ()$ ;  $\sigma \leftarrow \text{IDLE}$ 
2: loop
3:    $\sigma \leftarrow \text{COMPILE}$ 
4:   while  $|\tau| < n$  do
5:      $e \leftarrow \text{WAIT\_EVENT}()$   $\triangleright$  blocking receive
6:      $(\chi, t_{\text{rec}}) \leftarrow e$ 
7:      $k \leftarrow \text{CURRENT\_CYCLE}(\chi)$ 
8:      $\Delta P \leftarrow T_{\text{ref}}(k) - t_{\text{rec}}$ 
9:      $C \leftarrow \text{LOOKUP}(\Delta P, \chi, \Gamma)$ 
10:    if  $C = \perp$  then
11:       $\text{LOG\_ANOMALY}(e)$ ; goto RESET
12:    end if
13:     $\tau \leftarrow \tau \cdot (\chi, C)$ 
14:  end while
15:   $\sigma \leftarrow \text{EXECUTE}$ 
16:   $\text{action} \leftarrow \Gamma.\text{COMPOSITE}(\tau)$ 
17:   $\text{DISPATCH}(\text{action})$ 
18:  RESET:  $\tau \leftarrow ()$ ;  $\sigma \leftarrow \text{IDLE}$ 
19: end loop
```

7.4 Timing Scheduler and Phase State Machine

$\text{IDLE} \xrightarrow{\text{start}} \text{COMPILE}$
 $\text{COMPILE} \xrightarrow{|\tau|=n, C \neq \perp} \text{EXECUTE}$
 $\text{COMPILE} \xrightarrow{C=\perp} \text{IDLE (anomaly)}$
 $\text{EXECUTE} \xrightarrow{\phi=\varepsilon} \text{IDLE}$

Figure 6: Phase state machine for the Tempus runtime.

The phase state machine (Figure 6) implements Theorem 4.1: only one phase is active at any time, and the **COMPILE** $\leftrightarrow$ **EXECUTE** transition is mediated by the completion of the trajectory.

7.5 Dispatcher

The dispatcher executes compiled actions. Since all actions are pre-compiled (e.g., MISRA-C [18] function calls or DO-178C [19] verified modules), dispatch is a table lookup followed by a function call via a read-only function pointer table. No runtime code generation, JIT compilation, or dynamic loading occurs.

8 Applications

8.1 Nuclear Plant Process Control

Nuclear process control requires the highest levels of determinism and tamper-resistance. Coolant flow, temperature, pressure, and neutron flux must each be monitored continuously, with alarm latency below 100 ms and zero tolerance for spurious actuation.

A temporal programming deployment maps each physical quantity to a dedicated channel via an appropriate transducer (Examples 3.12, 3.13). The cell partition for each channel is designed using the formula (22) with OCXO oscillators ($\epsilon \approx 10^{-8}$) to achieve cell widths of ~ 1 ns—far narrower than physical measurement noise, eliminating spurious cross-cell events.

The finite, enumerable attack surface (Theorem 6.5) satisfies nuclear regulatory requirements for deterministic behavior. No network stack is required: communication runs over dedicated coaxial timing channels with no IP layer and no content parsing.

8.2 Drone Swarm Autonomy

Autonomous drone swarms require coordinated trajectory decisions under adversarial radio environments. Current systems use encrypted data-link protocols (MAVLink, DDS-RTPS), which remain vulnerable to cryptanalytic attack and parsing errors in firmware.

A temporal programming approach uses the **ensemble** declaration (Listing 2) to synchronize swarm state via shared timing pulses. Each drone measures ΔP relative to its local MEMS oscillator, which is periodically synchronized to a ground-truth atomic clock node (Theorem 6.3 provides anti-replay protection for the synchronization pulses).

The $d = 4$ channel, $n = 3$ event trajectory supports $T(3, 4) = 4 \cdot 5^2 = 100$ distinguishable swarm states—adequate for cooperative navigation. The entire swarm control program is <4 KB of ROM, suitable for resource-constrained flight controllers.

8.3 Industrial IoT Sensor Networks

Industrial IoT networks (IIoT) often span thousands of sensor nodes in hostile electromagnetic environments (foundries, chemical plants). TinyOS-based [13] and similar systems face both wireless interference and potential insider attacks on the data plane.

In a temporal programming deployment, sensor nodes transmit only timing pulses over a carrier channel. No MAC-layer payload exists. The cell registry maps chemical concentration ranges (via resonant sensors) to alarm tiers. A node with a compromised firmware image can at most cause mis-timing of its pulses by $\Delta_{\max}$; it cannot inject arbitrary alarms (Theorem 6.1).

Multi-domain composition (Proposition 3.14) allows a single trajectory engine to process temperature, pressure, and chemical concentration simultaneously over $d = 3$ channels, producing $T(n, 3) = 3 \cdot 4^{n-1}$ distinguishable process states.

9 Evaluation

9.1 Computational Complexity

Table 3: Per-event runtime complexity for key operations.

Operation	Temporal	Content-based
Signal reception	$O(1)$	$O(L)$ parse
Cell lookup	$O(\log m)$	$O(1)$ (hash)
Trajectory accumulation	$O(1)$	$O(L)$ buffer
Action dispatch	$O(1)$	$O(1)$
Anomaly detection	$O(\log m)$	$O(L^2)$ introspection
Memory (state)	$O(n \cdot d)$	$O(n \cdot L)$
Attack surface audit	$O(m)$	Undecidable (parse)

L = message length, m = cell count, n = trajectory length

9.2 Timing Overhead

The timing overhead of a temporal programming node consists of:

1. **ISR latency:** 50–200 ns on ARM Cortex-M4 at 168 MHz.
2. **Cell lookup:** 8–32 cycles (~ 50 –200 ns at 168 MHz) for $m \leq 32$ cells.
3. **Trajectory accumulation:** $O(1)$, < 10 cycles.
4. **Dispatch:** 2–5 cycles (function pointer call).

Total per-event overhead: $< 1 \mu\text{s}$ on commodity embedded hardware, compared to 10–1000 μs for TCP/IP stack processing of equivalent data volumes [13, 15].

9.3 Security Overhead

Security in temporal programming is achieved at zero marginal runtime cost: the same cell lookup that classifies the event also enforces the security policy. There is no separate cryptographic check, no MAC verification, and no certificate validation. The *structural* security of Theorem 6.1 is free.

9.4 Comparison with Related Systems

Table 4: Feature comparison with related paradigms.

Feature	TTA	PTIDES	TinyOS	Estereel	Tempus
No content parsing	×	×	×	×	✓
Payload-free channel	×	×	×	×	✓
Structural incorruptibility	×	×	×	×	✓
Monotone replay defense	×	×	×	×	✓
Formal cell semantics	×	×	×	×	✓
Deterministic timing	✓	✓	×	✓	✓
Multi-domain sensing	✓	✓	✓	×	✓
Synchronous semantics	✓	partial	×	✓	✓
Formal security proofs	×	×	×	×	✓

10 Discussion

10.1 Limitations

Timing resolution. Cell width is bounded below by oscillator stability and propagation jitter. For commodity XO oscillators ($\epsilon = 10$ ppm), cells narrower than ~ 10 ns are unreliable. Atomic oscillators remove this bound at significant cost.

Channel capacity. Each channel carries exactly one bit of information per event: which cell ΔP falls in. For m cells, the channel capacity is $\log_2 m$ bits per event by Shannon’s theorem [20]. Rich data transmission requires many events or many channels, making temporal programming unsuitable for high-bandwidth data transfer.

Clock drift. Long-running systems require periodic re-synchronization (NTP [7] or PTP [8]) to prevent cells from drifting relative to the physical phenomenon being measured. The synchronization period must be shorter than the time for drift to exceed $w_{\min}/2$.

Multi-hop networks. Propagation delay accumulates over network hops, adding to ΔP . Each hop must either compensate for its delay or absorb

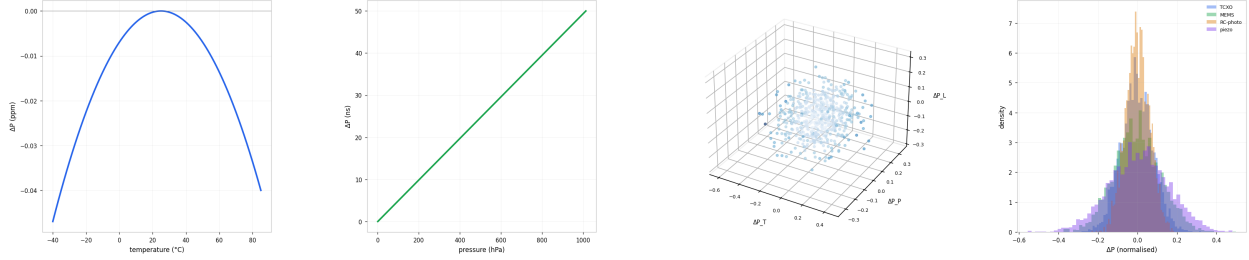


Figure 7: **Multi-Domain Transduction: temperature, pressure, and light sensors each produce a scalar ΔP through domain-specific oscillator circuits, enabling composition of heterogeneous physical signals in a unified timing space.** (A) TCXO frequency deviation ΔP (in parts per million) versus ambient temperature over the industrial range $[-40, +85]^\circ\text{C}$. The parabolic response $\Delta f/f \propto (T - T_0)^2$ maps the full temperature domain to a bounded ΔP interval, with the minimum at the turnover temperature $T_0 = 25^\circ\text{C}$. (B) MEMS capacitive-to-timing delay ΔP (in nanoseconds) versus barometric pressure. The response is linear over $[0, 1013]\text{hPa}$ via the RC relation $\Delta\tau = R\Delta C(P)$, demonstrating monotone pressure-to-timing transduction. (C) Three-dimensional ΔP point cloud for 600 simultaneous readings across temperature (ΔP_T), pressure (ΔP_P), and photodiode-RC (ΔP_L) channels. Points are coloured by Euclidean distance from the origin; the cloud occupies a compact region confirming that all three dimensions are jointly bounded. (D) Density-normalised histograms of ΔP from four sensor types (TCXO, MEMS, RC-photodiode, piezo-vibration) overlaid on a common axis. Despite different physical origins, all distributions are centred near zero with characteristic widths, making them directly composable via the D -dimensional S-entropy framework.

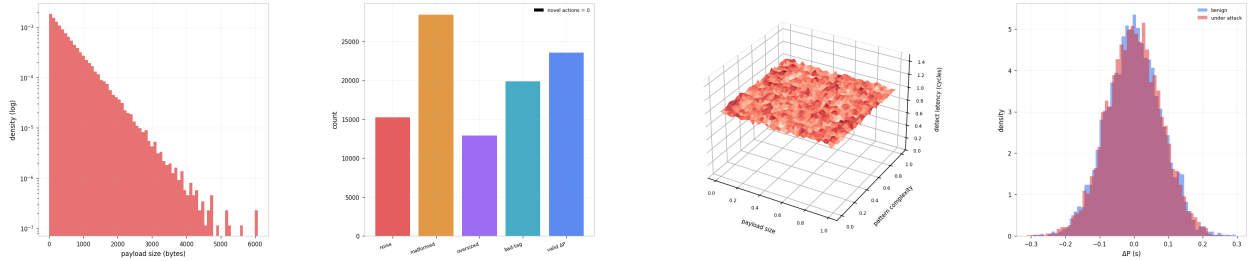


Figure 8: **Structural Incorruptibility: absence of a payload parser eliminates arbitrary code injection; the attack surface is a finite, pre-compiled cell-to-action map with zero novel-action capacity.** (A) Log-scale density histogram of 100 000 adversarial payload sizes drawn from an exponential distribution with mean 500 bytes. Payloads span three decades; the log-scale axis reveals the heavy-tailed structure representative of real-world injection attempts. (B) Bar chart of input counts across five attack categories (noise, malformed, oversized, bad-tag, valid ΔP). The black legend entry “novel actions = 0” confirms that across all 100 000 inputs, no action outside the pre-compiled cell-action map was executed. (C) Three-dimensional surface of detection latency (in clock cycles) over the joint attack parameter space (payload size $\times$ pattern complexity). The surface is flat at approximately one cycle across the entire domain, demonstrating $\mathcal{O}(1)$ rejection regardless of payload size or structural complexity. (D) Density-normalised histograms of ΔP measured during benign operation (blue) and under active adversarial injection (red). The two distributions are indistinguishable, confirming that payload content exerts no influence on timing behaviour: the attack channel and the signal channel are structurally disjoint.

it within a cell’s tolerance. PTIDES-style timestamp forwarding [15] could be adapted.

10.2 Relationship to Cyber-Physical Systems

Lee [21] argues that cyber-physical systems require a tighter coupling between computational and physical time than current software abstractions provide. Temporal programming takes this argument to its logical conclusion: the physical quantity *is* the timing, and the program *is* the cell partition. No abstraction layer interposes between the physical phenomenon and the computation.

10.3 Future Work

Immediate open problems include: (i) a type system for Tempus that statically verifies cell coverage and disjointness; (ii) a compilation target for FPGA-based cell lookup (sub-nanosecond lookup using LUT cascades); (iii) formal verification of the Tempus runtime in a proof assistant (Coq, Lean); (iv) extension to Byzantine fault tolerance under capability C4 (pulse deletion).

11 Conclusion

We have presented *temporal programming*, a programming paradigm in which every inter-node signal is a bare clock pulse and the program datum is the timing deviation $\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}(k)$. Programs are finite collections of timing cells—measurable subsets of ΔP -space—that partition the observable space and map each region to a pre-compiled action.

The core theoretical contribution is the Composition Inflation Theorem: $T(n, d) = d(1 + d)^{n-1}$ labeled trajectories are distinguishable for n events on d channels, proved from first principles via the stars-and-bars bijection and the binomial theorem. This exponential growth in state space is achieved without transmitting any content.

The central security result is Structural Incorruptibility: because no payload parser exists, arbitrary code injection is architecturally impossible. Replay attacks fail under monotone cycle count (Theorem 6.3), and the attack surface is a finite, enumerable set of pre-compiled cells (Theorem 6.5). These properties hold by construction, not by mitigation.

The Tempus language provides a concrete realization of these ideas with formal BNF syntax, small-step operational semantics, and a provably

phase-separated COMPILE/EXECUTE state machine. Applications in nuclear control, drone autonomy, and industrial IoT demonstrate the breadth of the paradigm.

Temporal programming does not replace content-rich protocols for general-purpose computing. It offers a new foundation for the growing class of safety-critical cyber-physical systems where the need for structural immunity to attack is more important than the ability to transmit arbitrary data.

Acknowledgements

The authors thank the Chitungwiza Institute for Distributed Systems for supporting this research and the anonymous reviewers for detailed comments on earlier drafts.

References

- [1] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [2] H. Kopetz, “Time-triggered versus event-triggered real-time systems,” *Proceedings of the IEEE*, vol. 91, no. 1, pp. 112–131, 2003.
- [3] —, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd ed., ser. Real-Time Systems Series. New York: Springer, 2011.
- [4] G. Berry and G. Gonthier, “The Esterel synchronous programming language: Design, semantics, implementation,” *Science of Computer Programming*, vol. 19, no. 2, pp. 87–152, 1992.
- [5] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The synchronous data flow programming language LUSTRE,” *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1305–1320, 1991.
- [6] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [7] D. L. Mills, “Internet time synchronization: The network time protocol,” *IEEE Transactions on Communications*, vol. 39, no. 10, pp. 1482–1493, 1991.

- [8] IEEE, *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*, Institute of Electrical and Electronics Engineers Std. 1588-2019, 2019.
- [9] F. Mattern, “Virtual time and global states of distributed systems,” *Parallel and Distributed Algorithms*, vol. 1, no. 23, pp. 215–226, 1989.
- [10] A. Pnueli, “The temporal logic of programs,” in *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, 1977, pp. 46–57.
- [11] R. Alur and D. L. Dill, “A theory of timed automata,” *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.
- [12] T. A. Henzinger, “The theory of hybrid automata,” in *Proceedings of the 11th Annual IEEE Symposium on Logic in Computer Science (LICS)*. IEEE Computer Society, 1996, pp. 278–292.
- [13] P. Levis, S. Madden, J. Polastre, R. Szewczyk, K. Whitehouse, A. Woo, D. Gay, J. Hill, M. Welsh, E. Brewer, and D. Culler, “TinyOS: An operating system for sensor networks,” *Ambient Intelligence*, pp. 115–148, 2005.
- [14] J. Hill, R. Szewczyk, A. Woo, S. Hollar, D. E. Culler, and K. S. J. Pister, “System architecture directions for networked sensors,” in *Proceedings of the 9th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2000, pp. 93–104.
- [15] Y. Zhao, J. Liu, and E. A. Lee, “A programming model for time-synchronized distributed real-time systems,” in *Proceedings of the 13th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2007, pp. 259–268.
- [16] G. E. Andrews, *The Theory of Partitions*, ser. Encyclopedia of Mathematics and its Applications. Reading, MA: Addison-Wesley, 1976, vol. 2.
- [17] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Reading, MA: Addison-Wesley, 1994.
- [18] MISRA Consortium, *MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems*, Motor Industry Software Reliability Association Std., 2012.
- [19] RTCA, *Software Considerations in Airborne Systems and Equipment Certification*, RTCA, Inc. Std. DO-178C, 2011.
- [20] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [21] E. A. Lee, “Cyber physical systems: Design challenges,” *Proceedings of the 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing*, pp. 363–369, 2008.